The **Hadoop Distributed File System (HDFS)** is the primary storage system used by the Hadoop framework for storing large datasets.

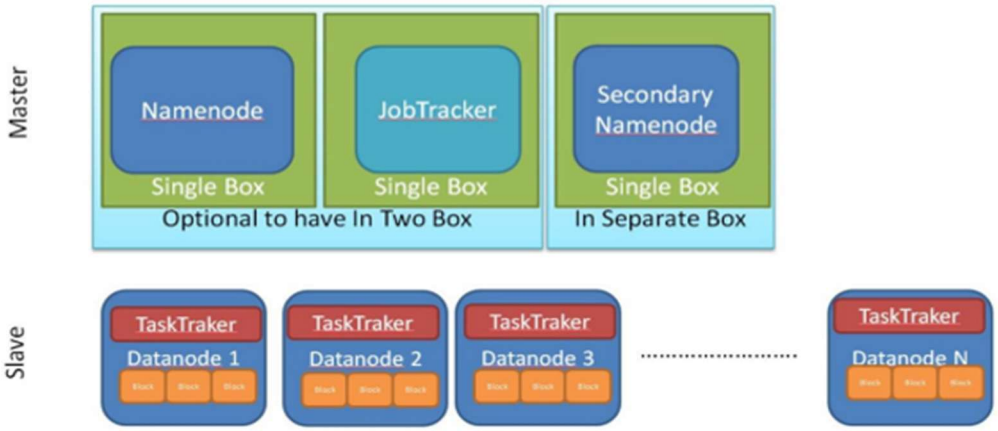
The **Hadoop Distributed File System (HDFS)** is the primary storage system used by the Hadoop framework for storing large datasets.

- HDFS stores very large files across multiple machines in a distributed environment and allows parallel processing of data. It is designed specifically for **big data applications**, where datasets can range from **gigabytes to petabytes** in size.

HDFS follows a **master–slave architecture** consisting of:

- **NameNode** – manages metadata and file system namespace
- **DataNodes** – store actual data blocks

This architecture enables efficient storage and retrieval of large-scale data.



Need for a Distributed File System

Traditional file systems store data on a single machine. However, modern applications generate massive volumes of data that exceed the storage and processing capacity of a single system.

A **distributed file system** solves this problem by distributing data across multiple machines connected in a network.

The main reasons for using a distributed file system are:

1. **Large Data Storage**
   Large datasets can be divided and stored across multiple machines.
2. **Parallel Processing**
   Data stored on different machines can be processed simultaneously.
3. **Reliability**
   Data is replicated across multiple nodes, ensuring availability even if some machines fail.
4. **Scalability**
   Storage capacity can be increased by adding more machines to the cluster.

HDFS provides all these capabilities, making it suitable for big data processing.

Feature	HDFS (Hadoop Distributed File System)	LFS (Local File System)	Cloud Object Storage
Definition	Distributed file system used in Hadoop clusters	File system on a single computer	Storage service that stores data as objects in the cloud
Storage Location	Multiple machines in a Hadoop cluster	Single local machine	Remote cloud servers
Data Structure	Files split into blocks	Files stored directly in folders	Data stored as objects with metadata
Data Size	Designed for very large datasets (TBs, PBs)	Suitable for small to medium files	Suitable for massive scalable storage
Fault Tolerance	High (block replication across nodes)	Low (single disk failure can lose data)	Very high (multiple cloud replicas)
Scalability	Scales by adding cluster nodes	Limited to single machine capacity	Almost unlimited scalability
Access Method	Hadoop APIs and distributed processing frameworks	Direct OS file access	Accessed via internet APIs
Performance	High throughput for big data processing	Low latency for local access	Depends on network latency
Example Systems	Hadoop Distributed File System	Windows NTFS, Linux ext4	Amazon S3, Google Cloud Storage

Design Features of HDFS

1. Very Large Files

HDFS is designed to store **very large files**, often ranging from hundreds of megabytes to terabytes in size.

Files are divided into **blocks**, which are stored across different machines in the cluster. This allows a file to be larger than the storage capacity of any single machine.

For example, a **1 TB file** can be split into many blocks and distributed across several DataNodes.

Many Hadoop clusters today store **petabytes of data**, making HDFS ideal for large-scale storage.

---

2. Streaming Data Access

HDFS is optimized for **streaming data access** rather than random access.

It follows a **write-once, read-many-times model**, which means:

- Data is written once into the file system
- The same data can be read many times for analysis

This design is well suited for applications like:

- data analytics
- machine learning
- log processing

Streaming access ensures that large files can be processed efficiently.

---

3. Commodity Hardware

HDFS is designed to run on **commodity hardware**, which refers to inexpensive and commonly available machines.

Unlike traditional systems that rely on expensive and highly reliable hardware, Hadoop clusters use:

- standard servers
- low-cost storage devices
- widely available components

Because such hardware may fail frequently, HDFS includes mechanisms for **fault tolerance and data replication**.

This approach significantly reduces infrastructure cost.

---

4. Fault Tolerance in HDFS

Hardware failures are common in large clusters. HDFS is designed to continue functioning even when machines fail.

Fault tolerance in HDFS is achieved through **data replication**.

Each file block is stored on multiple machines (typically **three replicas**).

Example:

Block A → Node1
Block A → Node2
Block A → Node3

If one machine fails, the system can still retrieve the block from another machine.

The **NameNode continuously monitors DataNodes** through heartbeat signals and re-replicates data if a node fails.

This ensures **high reliability and data availability**.

---

5. Scalability

HDFS is highly scalable.

New nodes can be added to the cluster without affecting existing operations.

This allows the storage capacity and processing power of the system to grow easily.

For example:

- A cluster may start with **10 nodes**

- Later it can be expanded to **100 or 1000 nodes**

This ability to scale horizontally makes HDFS suitable for big data environments.

Limitations of HDFS

Although HDFS is powerful, it is not suitable for certain types of applications.

---

1. Low Latency Data Access

HDFS is designed for **high-throughput batch processing**, not for applications requiring low latency.

Applications that require very fast responses (in milliseconds) such as:

- online transaction systems
- real-time databases

do not perform well on HDFS.

---

2. Lots of Small Files

HDFS is optimized for large files rather than many small files.

The **NameNode stores metadata about every file and block in memory**.

If there are too many small files:

- metadata size increases
- NameNode memory usage grows
- system performance decreases

Therefore, HDFS works best when storing fewer large files.

---

3. No Multiple Writers

HDFS allows **only one writer per file** at a time.

Multiple clients cannot write to the same file simultaneously.

This design simplifies consistency management but limits certain collaborative applications.

---

4. No Random File Modifications

Files in HDFS follow an **append-only model**.

data can only be written at the **end of the file**

- modifications at arbitrary positions are not supported

This restriction helps maintain data consistency and simplifies distributed storage management.

Blocks in HDFS

In HDFS, files are not stored as a single unit. Instead, they are divided into **smaller fixed-size units called blocks**.

For example:

If a file size is **256 MB** and the block size is **64 MB**, the file will be divided into **4 blocks**.

File → Block1 | Block2 | Block3 | Block4

Each block is stored independently on different machines (DataNodes) in the cluster.

Default Block Size

The default block size in HDFS is:

- **64 MB** (older Hadoop versions)
- **128 MB** (modern Hadoop installations)

Large block sizes help reduce the overhead caused by disk seek operations.

Example:

File Size = 300 MB
Block Size = 128 MB

Blocks Created:
Block1 → 128 MB
Block2 → 128 MB
Block3 → 44 MB

Replication Factor

The **replication factor** specifies how many copies of a block should be stored.

Default replication factor:

Replication Factor = 3

Example:

If a file has **2 blocks**, and replication factor is **3**, total blocks stored will be:

2 blocks × 3 replicas = 6 stored blocks

Replication ensures **high data availability and fault tolerance**.

Why HDFS Blocks Are Large

HDFS blocks are much larger than traditional disk blocks.

The reason is to **minimize disk seek time**.

Disk operations involve two steps:

1. **Seek time**
   Time taken for the disk head to reach the location of data.
2. **Data transfer time**
   Time taken to read the actual data.

If blocks are too small:

- many seek operations occur
- performance decreases

By using large blocks:

- fewer seek operations occur
- data transfer happens efficiently

Seek Time ≈ 10 ms
Transfer Rate ≈ 100 MB/s

To make seek time only **1% of transfer time**, block sizes around **100 MB** are ideal.

Therefore Hadoop uses: Block Size = 64 MB or 128 MB

Block A replicas:

Node1 (Rack 1)
Node2 (Rack 1)
Node5 (Rack 2)

So the replicas are stored:

- two copies in **Rack 1**
- one copy in **Rack 2**

Why This Placement Is Important

Case 1 — Node Failure

If **Node1 fails**, the block can still be read from **Node2 or Node5**.

Case 2 — Entire Rack Failure

If **Rack 1 fails**, the block can still be accessed from **Rack 2 (Node5)**.

This ensures **data reliability**.

Command-Line Interface (CLI)

The **Command-Line Interface (CLI)** in Hadoop allows users to interact with the **Hadoop Distributed File System (HDFS)** using commands through the terminal or shell.

These commands are similar to **Linux filesystem commands**, but they operate on files stored in HDFS.

The general format of an HDFS command is:

hdfs dfs <command> <path>

hdfs dfs -ls
hdfs dfs -mkdir
hdfs dfs -put
hdfs dfs -get
hdfs dfs -cat
hdfs dfs -rm
hdfs dfs -du
hdfs dfs -df
hdfs dfs -copyFromLocal
hdfs dfs -copyToLocal

Example:

hdfs dfs -put file.txt /input

Uploads a file to HDFS.

Hadoop Filesystems

Hadoop supports multiple types of filesystems through a unified interface called the **Hadoop FileSystem abstraction**.